

Conventional Commits

Guía de referencia rápida para la estandarización de mensajes en Git

La especificación de **Conventional Commits** es una convención ligera sobre los mensajes de los commits que proporciona un conjunto fácil de reglas para crear un historial de commits explícito. Esto facilita la creación de herramientas automáticas (como la generación de CHANGELOGs o el versionado semántico).

Estructura del Mensaje

El mensaje del commit debe tener la siguiente estructura estructural:

```
<tipo>[ámbito opcional]: <descripción>
```

```
[cuerpo opcional]
```

```
[nota o notas al pie opcionales]
```

Prefijos más Utilizados

PREFIJO	¿CUÁNDO SE UTILIZA?	EJEMPLO PRÁCTICO
feat:	Cuando se introduce una nueva funcionalidad al código (corresponde con MINOR en el versionado semántico).	feat: añadir botón de login con Google
fix:	Cuando se soluciona un fallo o error (bug) en la aplicación (corresponde con PATCH en el	fix: corregir parpadeo en el menú lateral

PREFIJO	¿CUÁNDO SE UTILIZA?	EJEMPLO PRÁCTICO
	versionado semántico).	
docs:	Para cambios que afecten únicamente a la documentación (archivos README, manuales, comentarios de código).	docs: actualizar guía de instalación en Linux
style:	Cambios que no afectan al significado del código (espacios en blanco, formateo, punto y coma omitidos, etc.).	style: formatear con Prettier y corregir sangrías
refactor:	Modificaciones en el código que ni corrigen un error ni añaden una funcionalidad , sino que mejoran su estructura interna.	refactor: simplificar bucle de validación de usuarios
perf:	Cambios en el código orientados estrictamente a **mejorar el rendimiento** o la velocidad de la aplicación.	perf: optimizar consulta SQL de productos
test:	Para añadir pruebas que	test: añadir pruebas unitarias para el carrito

PREFIJO	¿CUÁNDO SE UTILIZA?	EJEMPLO PRÁCTICO
	faltaban o **corregir/ modificar tests** existentes. No altera el código de producción.	
chore:	Tareas rutinarias de mantenimiento, actualización de dependencias, o configuraciones de herramientas que no cambian el código fuente.	chore: actualizar versión de la librería Axios
ci:	Cambios en los archivos y scripts de **Integración Continua / Despliegue Continuo** (ej. GitHub Actions, GitLab CI, Jenkins).	ci: añadir paso de linter al flujo de trabajo

Buenas Prácticas Rápidas

- **Usa el imperativo:** Escribe la descripción en tiempo imperativo y presente (ej: "añadir" en vez de "añadido" o "añade").
- **Minúsculas:** El prefijo y la primera palabra de la descripción suelen ir en minúsculas.
- **Sin punto final:** No pongas un punto . al terminar la frase de la descripción.
- **Cambios disruptivos (Breaking Changes):** Si el cambio rompe la compatibilidad hacia atrás, añade una exclamación justo antes de los dos puntos (ej:
feat!: cambiar endpoint de la API pública) o detállalo en el cuerpo como
BREAKING CHANGE: .

